

박재현

Software Engineer

EMAIL | wogusqkr0515@gmail.com
PHONE | 010-2488-3413
GITHUB | github.com/wogusqkr0515

MES/ERP/TQM 시스템을 개발하고 운영하는 소프트웨어 엔지니어입니다.

운영 시스템의 가시성 확보, 장애 원인 분석, 메시징/DB 병목 진단 등 안정적인 서비스 운영을 위한 개선 업무를 수행했습니다.

브라질, 영국, 태국, 중국, 인도 현장 출장과 해외 엔지니어 협업을 통해 글로벌 운영 환경에서 문제를 조율한 경험이 있습니다.

EXPERIENCE

2026.06	MES 운영 모니터링 구축 Prometheus, Grafana, 운영 지표 설계 국내 KCC 6개 공장 MES는 장애 발생 후 서버별 상태를 직접 확인해야 해 원인 파악이 늦고, 3~4개월 주기로 재기동이 필요한 반복 오류도 근거 수집이 어려웠습니다. 서버/애플리케이션 지표와 업무 처리량, 오류 추이를 수집해 공장별로 비교할 수 있는 모니터링 화면을 구성했습니다. 반복 오류가 발생한 시점의 지표를 확인할 수 있게 되어 추측이 아닌 데이터 기반으로 원인을 분석할 수 있게 했습니다.
2026.05 - 현재	ERP 마이그레이션 Java 17, Spring Boot, WebSquare 5 기존 ERP는 Spring 2.1/WebLogic 기반으로 구성되어 실행 환경과 트랜잭션 처리 방식이 신규 표준과 달랐고, 단순 버전 교체만으로는 안정적인 전환이 어려웠습니다. Tomcat/Spring Boot 구조로 옮기면서 다중 DB 처리, 공통 모듈, 화면-서비스-DB 간 연결 방식을 점검하고 수정 대상을 정리했습니다. 기존 업무 기능이 신규 환경에서도 동일하게 동작하도록 마이그레이션을 진행하고 있습니다.
2025.09 - 2026.04	글로벌 TQM 품질관리 시스템 구축 Spring Boot, MSSQL, JPA, GitLab 6개국에 배포되는 품질 시스템으로, 같은 기능 안에서도 국가별 검사 기준, 코드 체계, 화면 표현 방식이 달랐습니다. DCS/8D/QAA/Audit/PPAP 등 6개 모듈의 백엔드를 개발하며 공통 업무 로직과 국가별 설정값을 분리했습니다. 중국/일본 1차 오픈을 지원했고, 미국/유럽 확장 시 현지 요구사항을 코드 수정 없이 설정 변경으로 반영할 수 있게 했습니다.
2025.04 - 2025.08	글로벌 MES 신규 구축 및 안정화 Spring Boot, MSSQL, WebSquare, JavaScript 브라질/영국/태국 3개국 5개 공장 MES 오픈 과정에서 현장 사용자 문의, 기능 오류, 데이터 입력 방식 차이가 동시에 발생했습니다. 현업 문의를 개발 이슈로 정리하고 우선순위를 조율했으며, 국가별 숫자 포맷 차이로 발생한 입력 오류를 공통 인터셉트 로직으로 처리했습니다. Go-Live 현장에서 오류 대응과 사용자 지원을 함께 수행하며 초기 운영 안정화를 지원했습니다.
2025.04 - 2025.05	실시간 모니터링 시스템 자체 개발 Spring Boot, WebSocket, MSSQL, Highcharts 기존 실시간 데이터 관제는 외부 솔루션 의존도가 높아 라이선스 비용 부담이 크고 필요한 화면을 빠르게 추가하기 어려웠습니다. WebSocket 스트리밍과 부분 갱신 방식으로 생산 데이터 대시보드를 자체 구현했고, 수집 주기와 화면 반영 속도를 조정해 현장에서 주요 설비 상태를 실시간으로 확인할 수 있게 했습니다. AVEVA PI System을 대체해 라이선스 비용을 약 80% 절감했습니다.
2024.12 - 2025.02	C#/.NET 레거시 MES 마이그레이션 C# 분석, Spring Boot, MSSQL, WebSquare 12년간 운영된 C#/.NET MES는 문서가 부족해 장애 대응과 신규 기능 반영 시 기존 로직을 파악하는 데 시간이 많이 들었습니다. 화면, 배치, API 흐름을 역분석해 프로세스 다이어그램과 인터페이스 문서를 만들고, 기능 단위로 모듈을 재정리해 Spring Boot 기반으로 재구현했습니다. 이후 같은 유형의 기능을 더 빠르게 반영할 수 있게 되어 신규 기능 리드타임을 약 80% 단축했습니다.
2022.03 - 2022.08	부산은행 마이데이터 API Oracle, MyBatis, Java 마이데이터 재제공/기관 간 전송 API는 기관별 요청 구조가 유사했지만 검증과 응답 처리 로직이 반복되어 신규 API 개발과 결합 확인에 시간이 걸렸습니다. 반복 로직 50건 이상을 공통 모듈로 분리하고 유효성 검증 기준을 표준화했습니다. Oracle 인덱스와 MyBatis 배치 처리도 함께 개선해 응답 지연을 줄이고 API 개발 리드타임을 단축했습니다.

SKILLS

Backend: Java, Spring Boot, JPA/MyBatis

Database: MSSQL, Oracle

Monitoring/Operation: Prometheus, Grafana

EDUCATION / CERTIFICATIONS

부산대학교 정보컴퓨터공학부 컴퓨터공학 전공, 2018.03 - 2024.08

부산대 소프트웨어공학연구실 MSA 의존성 분석 연구, 2022.10 - 2023.09

BCE 패턴 기반의 마이크로서비스 결합 추천 시스템, 2023 KCC 학부생부문 1저자

정보처리기사, 네트워크관리사 2급, TOEIC Speaking AL

Project 01

MES 운영 모니터링 구축

PERIOD	2026.06
SCOPE	국내 KCC 6개 공장 MES
STACK	Prometheus, Grafana

장애 이후 수동 확인하던 운영 상태를 데이터로 추적할 수 있게 만든 프로젝트

기존 MES 운영은 장애 신고가 들어온 뒤 담당자가 서버별 로그와 상태를 직접 확인하는 방식에 가까웠습니다. 특히 3~4개월 주기로 재기동이 필요했던 반복 오류는 원인 후보가 여러 계층에 걸쳐 있었지만, 발생 시점의 근거 지표가 부족해 분석이 늦어졌습니다.

문제 장애 시점의 서버, JVM, 업무 처리 상태를 한곳에서 비교하기 어려움

초점 원인 추적에 필요한 최소 지표부터 수집하고 공장별 비교 화면 구성

성과 반복 오류와 처리 지연을 데이터 기반으로 분석할 수 있는 운영 화면 확보

PROBLEM 국내 6개 공장에서 사용하는 MES는 장애가 발생하면 사용자가 먼저 현상을 신고하고, 이후 운영자가 각 서버와 애플리케이션 로그를 확인하는 흐름이었습니다. 장애가 이미 지나간 뒤에는 당시 CPU, 메모리, GC, thread, DB pool, 메시지 처리 상태를 한 번에 확인하기 어려웠습니다.

특히 일부 MES 프로세스는 3~4개월 주기로 재기동이 필요했지만, 장애가 발생한 시점의 지표가 남아 있지 않아 원인을 특정하기 어려웠습니다. 현상은 애플리케이션 멈춤처럼 보였지만 실제 원인은 DB 대기, 메시지 적체, JVM 메모리 사용량, 특정 업무 요청 집중 등 여러 후보로 나뉠 수 있었습니다.

결과적으로 오류가 반복되어도 특정 서버 문제인지, 업무 요청 증가인지, DB 대기인지, 메시지 처리 지연인지 판단하는 데 시간이 걸렸고, 장애 대응도 담당자의 경험과 수동 확인에 의존하는 비중이 컸습니다.

DIFFICULTY 단순 서버 리소스만 수집하면 실제 업무 장애 원인을 설명하기 어려웠습니다. MES는 여러 JVM과 메시징 구간, DB connection pool, 공장별 업무 피크가 함께 얹혀 있었기 때문입니다.

예를 들어 CPU 사용률이 낮아도 DB connection pool이 소진되면 업무 처리는 멈출 수 있고, JVM은 살아 있어도 특정 worker가 오래 점유되면 후속 요청이 지연될 수 있었습니다. 그래서 서버가 살아 있는지보다, 업무 요청이 정상적으로 흐르고 있는지를 보여주는 지표가 필요했습니다.

또한 운영 시스템에 과도한 변경을 넣을 수 없어서, 먼저 부담이 낮은 지표부터 수집하고 업무 지표는 검증된 구간에 단계적으로 추가해야 했습니다.

DECISION 처음부터 모든 업무 로직을 계속하기보다, 장애 원인 후보를 좁히는 데 필요한 지표를 우선순위로 정했습니다. 서버 리소스, JVM 상태, DB pool 사용량, 메시지 처리량, 오류 추이를 공장별 프로세스 단위로 비교할 수 있게 잡았습니다.

지표는 단순히 많이 수집하는 것이 아니라 장애가 발생했을 때 질문에 답할 수 있어야 했습니다. 어느 공장에서 먼저 문제가 시작됐는지, 특정 JVM만 비정상인지, DB 대기가 늘어난 시점과 업무 오류가 맞물리는지 확인할 수 있도록 화면을 나눴습니다.

이 방식은 운영 영향도를 낮추면서도 장애 시점에 가장 먼저 확인해야 할 정보를 확보할 수 있었습니다.

SOLUTION 공장별 MES 프로세스를 기준으로 지표 수집 대상을 정리하고, 서버/애플리케이션/업무 처리 흐름을 나눠 모니터링 화면을 구성했습니다. 운영자는 공장별 처리량, 오류 발생 추이, JVM 상태, DB connection pool 사용 상태를 같은 화면에서 비교할 수 있게 했습니다.

LOTsvr 장애 분석에서는 특정 작업 이후 DB 대기가 길어지고 worker와 DB pool 대기가 누적되는 흐름을 지표와 로그로 함께 확인했습니다. 단순히 프로세스를 재기동하는 데서 끝내지 않고, 어떤 요청이 오래 점유되는지, 같은 LOT에 대한 중복 요청이 쌓이는지, connection wait이 업무 지연으로 이어지는지 확인할 수 있게 정리했습니다.

결과적으로 장애가 발생했을 때 담당자가 개별 서버를 순서대로 열어보는 방식에서 벗어나, 공장별 상태와 병목 후보를 먼저 비교한 뒤 로그를 확인하는 흐름으로 개선했습니다.

Project 02

실시간 모니터링 시스템 자체 개발

PERIOD	2025.04 - 2025.05
SCOPE	생산 데이터 실시간 관제
STACK	Spring Boot, WebSocket

외부 솔루션 의존도가 높던 실시간 관제를 자체 시스템으로 대체한 프로젝트

생산 데이터 실시간 관제는 현장 업무에 필요했지만, 기존 솔루션은 라이선스 비용 부담이 크고 필요한 화면을 빠르게 추가하기 어려웠습니다. 제한된 기간 안에 현장에서 바로 사용할 수 있는 대시보드를 만들기 위해 데이터 수집, 스트리밍, 화면 갱신 방식을 함께 설계했습니다.

문제	외부 솔루션 비용 부담과 화면 확장 제약
초점	실시간성, 화면 반영 속도, 운영자가 이해할 수 있는 지표 구성
성과	AVEVA PI System 대체, 라이선스 비용 약 80% 절감

PROBLEM 기존 실시간 데이터 관제는 외부 솔루션 의존도가 높았습니다. 이미 수집되는 생산 데이터를 현장 기준에 맞게 재구성하거나, 필요한 화면을 빠르게 추가하는 데 제약이 있었습니다.

현장에서는 설비 상태와 생산 흐름을 빠르게 확인해야 했지만, 범용 솔루션 화면은 실제 업무에서 보고 싶은 기준과 완전히 일치하지 않았습니다. 특정 생산 지표를 추가하거나 화면 구성을 바꾸려면 외부 솔루션의 제약을 먼저 고려해야 했고, 변화가 필요한 시점에 내부 개발팀이 바로 대응하기 어려웠습니다.

비용 측면에서도 계속 외부 솔루션에 의존하기보다, 내부 시스템에서 필요한 범위부터 직접 구현할 필요가 있었습니다.

DIFFICULTY 실시간 화면은 단순 조회 화면과 달리 데이터 수집 주기, 화면 갱신 주기, 연결 끊김, 일시적인 데이터 지연을 함께 고려해야 했습니다. 화면이 너무 자주 갱신되면 사용자가 상태를 읽기 어렵고, 너무 늦게 갱신되면 관제 화면의 의미가 줄어듭니다.

또한 데이터가 들어오는 속도와 사용자가 화면에서 받아들이는 속도는 달랐습니다. 모든 데이터를 그대로 전달하면 네트워크와 브라우저 부담이 커지고, 반대로 화면 갱신을 줄이면 현장에서는 실시간성이 부족하다고 느낄 수 있었습니다.

1인 개발로 진행했기 때문에 기능 범위를 명확히 줄이고, 현장에서 가장 먼저 필요한 지표를 우선 구현해야 했습니다.

DECISION 전체 솔루션을 한 번에 대체하기보다, 현장 사용자가 실제로 확인해야 하는 생산 지표와 화면부터 우선 구현했습니다. 서버는 Spring Boot 기반으로 구성하고, 화면에는 WebSocket 스트리밍과 부분 갱신 방식을 적용했습니다.

WebSocket을 선택한 이유는 사용자가 계속 새로고침하거나 화면이 주기적으로 전체 데이터를 다시 조회하는 방식을 피하기 위해서였습니다. 서버가 변경된 데이터를 선별해 전달하고, 화면은 필요한 영역만 갱신하면 실시간성을 유지하면서도 불필요한 조회를 줄일 수 있었습니다.

이 선택은 주기적 polling만 사용할 때보다 화면 반응성을 높이면서도, 변경된 데이터만 갱신해 화면 부담을 줄일 수 있었습니다.

SOLUTION 생산 데이터 수집 구간과 화면 표시 구간을 분리하고, 실시간으로 보여야 하는 지표를 대시보드 형태로 구성했습니다. 데이터가 들어오면 WebSocket으로 화면에 전달하고, 화면은 전체 재조화가 아니라 필요한 영역만 갱신하도록 설계했습니다.

수집된 데이터는 화면에서 바로 쓰기 좋은 관제 지표로 정리했고, 화면에는 상태 변화가 필요한 영역만 반영했습니다. 연결이 끊기거나 일시적으로 데이터가 늦게 들어오는 상황도 고려해 화면과 서버의 역할을 분리했습니다.

현장에서는 주요 설비와 생산 상태를 별도 솔루션 화면에 의존하지 않고 확인할 수 있게 되었고, 운영 중에도 필요한 지표를 내부 개발로 추가할 수 있는 형태가 되었습니다. 결과적으로 AVEVA PI System을 대체하며 라이선스 비용을 약 80% 절감했습니다.

Project 03

글로벌 TQM 품질관리 시스템 구축

PERIOD	2025.09 - 2026.04
SCOPE	6개 모듈, 6개국 배포
STACK	Spring Boot, MSSQL

국가별 품질 기준 차이를 공통 시스템 안에서 운영 가능하게 만든 프로젝트

글로벌 품질관리 시스템은 같은 품질 업무를 다루더라도 국가별 검사 기준, 코드 체계, 승인 흐름, 화면 표현 방식이 달랐습니다. 모든 차이를 코드로 처리하면 국가가 늘어날수록 유지보수가 어려워지기 때문에, 공통 업무 로직과 국가별 설정을 분리하는 방향으로 개발했습니다.

문제 국가별 품질 기준 차이가 같은 기능 안에서 반복 발생

초점 공통 업무 로직과 현지화 설정의 분리

성과 중국/일본 오픈 지원, 미국/유럽 확장 대응

PROBLEM TQM 시스템은 DCS, 8D, QAA, Audit, PPAP 등 여러 품질 업무를 하나의 시스템으로 묶는 프로젝트였습니다. 문제는 동일한 기능이라도 국가별로 코드 체계, 입력 기준, 담당 조직, 화면 문구가 달랐다는 점입니다.

예를 들어 품질 이슈를 등록하고 조치하는 큰 흐름은 같아도, 국가별로 필수 입력값, 결재 기준, 담당 부서, 코드값이 달라질 수 있었습니다. 초기 국가만 보고 기능을 만들면 다음 국가 확장 때마다 코드 수정이 반복될 가능성이 컸습니다.

따라서 프로젝트의 핵심은 단순히 화면과 API를 만드는 것이 아니라, 여러 국가가 같은 시스템을 쓰면서도 각 국가의 업무 기준을 반영할 수 있게 만드는 것이었습니다.

DIFFICULTY 모든 것을 설정으로 빼면 개발은 유연해 보이지만, 업무 규칙이 흐려지고 운영자가 이해하기 어려운 시스템이 될 수 있었습니다. 반대로 모든 것을 코드로 고정하면 국가별 차이를 반영할 때마다 수정 범위가 커졌습니다.

또한 한 국가의 예외를 급하게 코드에 넣으면 이후 다른 국가에서 같은 모듈을 사용할 때 예상하지 못한 분기가 늘어날 수 있었습니다. 품질 시스템은 등록, 검토, 조치, 승인, 이력 관리가 이어지는 흐름이라 작은 기준값 차이도 후속 단계에 영향을 줄 수 있었습니다.

따라서 어떤 규칙은 전사 공통으로 고정하고, 어떤 항목은 국가별 설정으로 남길지 판단하는 것이 핵심이었습니다.

DECISION 공통 업무 흐름은 백엔드 로직으로 유지하고, 국가별로 달라질 수 있는 기준값과 표시 방식은 설정으로 분리했습니다. 품질 모듈별로 상태, 코드, 첨부, 승인 흐름을 나눠 보면서 재사용 가능한 처리와 국가별 예외를 구분했습니다.

공통화 대상은 모듈의 생명주기와 데이터 정합성에 직접 연결되는 로직으로 잡고, 국가별로 달라질 수 있는 값은 기준 데이터와 설정으로 분리했습니다. 이렇게 나누면 현지 요구사항을 반영하더라도 핵심 로직을 매번 수정하지 않아도 됩니다.

이 방식은 중국/일본 1차 오픈 후 미국, 이탈리아, 독일, 영국 확장 시 같은 구조를 유지하면서 현지 요구사항을 반영할 수 있는 장점이 있었습니다.

SOLUTION DCS/8D/QAA/Audit/PPAP 등 6개 모듈의 백엔드를 개발하며 공통 코드, 기준 데이터, 국가별 설정값을 분리했습니다. 업무 흐름상 공통으로 유지해야 하는 저장, 조회, 상태 변경 로직은 모듈 기준으로 정리하고, 국가별 검사 기준과 화면 표현 차이는 설정값으로 처리했습니다.

중국과 일본 1차 오픈 과정에서는 현지 요구사항을 확인하며 설정과 로직의 경계를 조정했습니다. 특정 국가에서만 필요한 코드값이나 화면 문구는 설정으로 관리하고, 여러 국가가 동일하게 따라야 하는 상태 전환과 저장 규칙은 백엔드 로직으로 유지했습니다.

이후 미국과 유럽 확장에서는 기존 구조를 유지하면서 필요한 현지 요구사항을 반영할 수 있도록 개발했습니다. 결과적으로 국가가 늘어날 때마다 신규 분기를 추가하는 방식이 아니라, 공통 모듈 위에 국가별 기준을 얹는 방식으로 확장할 수 있게 했습니다.